Tarefa 11: Particionamento de Dados e Balanceamento de Carga com OpenMP

Matheus Bezeril Pinto

Outubro de 2026

Resumo

Este relatório apresenta uma simulação bidimensional de difusão viscosa implementada em C e paralelizada com OpenMP. O modelo retém apenas o termo viscoso da evolução da velocidade, utiliza diferenças finitas no espaço, Euler explícito no tempo e condições de contorno periódicas. A validação considera estados parado, uniforme e com perturbação gaussiana. Para a análise de desempenho, foram comparadas versões com e sem `collapse(2)`, utilizando os schedules `static`, `dynamic,8` e `guided,8`, com uma, duas e quatro threads solicitadas. Os testes utilizaram uma malha de 400×400 células, 5.000 passos e três repetições por configuração, totalizando 54 medições. O menor tempo médio foi de aproximadamente 1,762 s, obtido sem `collapse(2)`, com quatro threads e `guided,8`, correspondendo a um speedup de 3,267 sobre a referência comum. O uso de `collapse(2)` com `dynamic,8` apresentou custo elevado de escalonamento. Os resultados mostram que a granularidade do trabalho deve ser considerada junto à escolha do schedule.

1 Introdução

A paralelização de laços exige identificar quais iterações podem ser executadas de forma independente e como distribuir esse trabalho entre as threads. O OpenMP permite expressar essa distribuição por diretivas e cláusulas, como `schedule` e `collapse` [1].

Nesta tarefa, cada célula de uma malha possui duas componentes de velocidade, u e v. O estado seguinte de uma célula depende de seu valor atual e de quatro vizinhos. Embora exista dependência entre passos de tempo consecutivos, as células de um mesmo passo podem ser calculadas em paralelo quando os campos de entrada e saída são separados.

O objetivo é verificar a correção dessa evolução e comparar o efeito das diferentes distribuições de trabalho. A implementação foi mantida próxima à versão sequencial em `Old/`, preservando os modos de inicialização, os diagnósticos e a visualização em Python. Os fontes e os resultados analisados correspondem à revisão `5192499` do repositório [2].

2 Modelo e discretização

2.1 Difusão viscosa

Ao considerar somente os efeitos viscosos com viscosidade cinemática constante ν, o modelo utilizado é

$$\frac{\partial u}{\partial t} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \tag{1}$$

$$\frac{\partial v}{\partial t} = \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right). \tag{2}$$

Não há pressão, forças externas ou transporte advectivo. Assim, trata-se de um modelo de difusão das componentes da velocidade; o programa não resolve o sistema geral de Navier–Stokes nem inclui uma projeção para impor incompressibilidade.

O domínio é $[0, L_x) \times [0, L_y)$, com $L_x = L_y = 1$. Para N_x e N_y células, definem-se

$$\Delta x = \frac{L_x}{N_x}, \qquad \Delta y = \frac{L_y}{N_y}.$$

A velocidade é associada aos centros das células: $x_i = (i + 1/2)\Delta x$ e $y_j = (j + 1/2)\Delta y$.

2.2 Diferenças finitas e Euler explícito

Para uma componente genérica w, a atualização é

$$\begin{aligned} w_{i,j}^{n+1} = w_{i,j}^{n} &+ r_x \left(w_{i+1,j}^{n} - 2w_{i,j}^{n} + w_{i-1,j}^{n} \right) \\ &+ r_y \left(w_{i,j+1}^{n} - 2w_{i,j}^{n} + w_{i,j-1}^{n} \right), \end{aligned} \tag{3}$$

em que

$$r_x = \frac{\nu \Delta t}{\Delta x^2}, \qquad r_y = \frac{\nu \Delta t}{\Delta y^2}.$$

O esquema tem precisão de primeira ordem no tempo e segunda ordem no espaço. Para a difusão explícita em duas dimensões, a condição de estabilidade é

$$r_x + r_y \leq \frac{1}{2}.$$

O código define $\nu = 0{,}01$, $\alpha = 0{,}20$ e

$$\Delta t = \frac{\alpha \min(\Delta x^2, \Delta y^2)}{\nu}. \tag{4}$$

Consequentemente, $r_x + r_y \leq 2\alpha = 0{,}40$, atendendo à condição inclusive em malhas retangulares.

As condições de contorno são periódicas: o vizinho anterior ao índice zero é o último índice da dimensão, e o vizinho após o último índice é zero. O domínio não possui paredes nem uma fronteira de saída.

2.3 Condições iniciais

Os três modos de inicialização são:

1. Modo `0`: $u = v = 0$, representando o fluido parado.
2. Modo `1`: $u = 1$ e $v = 0$, representando velocidade uniforme.
3. Modo `2`: $v = 0$ e

$$u(x, y, 0) = 1 + 0{,}5 \exp\left[-\frac{(x - 0{,}5)^2 + (y - 0{,}5)^2}{2(0{,}08)^2}\right].$$

Nos dois primeiros modos, o Laplaciano é nulo e o estado deve permanecer constante. No terceiro, a gaussiana deve se espalhar e perder seu pico. A evolução por difusão não transporta o centro da perturbação como faria um termo advectivo.

3 Implementação

3.1 Organização dos arquivos e da memória

A implementação divide as responsabilidades em:

- `fluid.h`: parâmetros, estrutura `Simulation` e protótipos;
- `fluid.c`: alocação, inicialização, condições periódicas, evolução, diagnósticos e gravação;
- `fluid_sim.c`: seleção do modo e coordenação da execução;
- `visualize_fluid.py`: leitura dos CSVs e geração do GIF;
- `executar_testes.sh`: compilação e comparação das configurações OpenMP.

Os quatro vetores `u`, `v`, `u_new` e `v_new` são alocados com `calloc`. A posição (i, j) corresponde ao índice linear $iN_y + j$. Portanto, o laço interno em j percorre posições contíguas de memória.

Durante um passo, os vetores atuais são somente lidos e cada iteração escreve em uma posição distinta dos vetores novos. Não é necessário usar `atomic` ou `critical` nessa atualização. Para a carga de 400×400, os quatro campos ocupam aproximadamente 5,12 MB, sem contar estruturas e outros custos do processo.

3.2 Laços paralelos e troca dos campos

As duas versões são produzidas a partir do mesmo fonte:

Código 1: Seleção da diretiva OpenMP durante a compilação.

```
#ifdef USE_COLLAPSE
    #pragma omp parallel for collapse(2) schedule(runtime)
#else
    #pragma omp parallel for schedule(runtime)
#endif
    for (int i = 0; i < sim->nx; ++i)
    {
        for (int j = 0; j < sim->ny; ++j)
        {
            /* Leitura dos vizinhos e escrita da celula nova. */
        }
    }
```

O trecho acima resume a estrutura dos laços; o corpo completo está na cópia de `fluid.c` incluída na pasta `codigos/` do projeto do relatório.

Sem `collapse`, o espaço distribuído contém N_x iterações, cada uma responsável por uma linha com N_y células. Com `collapse(2)`, as duas dimensões formam um único espaço lógico com $N_x N_y$ iterações, cada uma correspondente a uma célula.

A opção `-DUSE_COLLAPSE` do GCC define a macro `USE_COLLAPSE` no pré-processador e seleciona a primeira diretiva. O prefixo `-D` é uma opção do compilador; não faz parte do nome da macro.

Ao final do `parallel for`, há uma barreira implícita. Somente depois que todas as células novas estão prontas, `swap_fields` troca os ponteiros dos campos atuais pelos novos. Essa troca evita copiar todos os valores e garante que o próximo passo leia um estado completo.

3.3 Schedules e granularidade

A cláusula `schedule(runtime)` seleciona a política em tempo de execução. Ela é definida pela variável `OMP_SCHEDULE`, sem recompilação:

- `static`: distribuição estática das iterações; não há obtenção dinâmica de novos blocos durante o laço;
- `dynamic,8`: as threads obtêm blocos de oito iterações conforme terminam o trabalho;
- `guided,8`: os blocos começam maiores e diminuem, com tamanho mínimo de oito, exceto um possível bloco final.

O trabalho por célula é essencialmente uniforme. Não há regiões em que uma célula execute um algoritmo mais caro que outra. Portanto, a distribuição dinâmica pode introduzir custo sem necessariamente corrigir um desequilíbrio relevante.

É importante interpretar a unidade do bloco: oito iterações significam oito linhas na versão normal, mas somente oito células na versão com `collapse(2)`.

3.4 Medição e modo de desempenho

A função `run_simulation` acumula o tempo apenas do cálculo e da troca dos campos:

Código 2: Trecho que delimita a medição em cada passo.

```
double inicio = omp_get_wtime();

compute_next_state(sim);
swap_fields(sim);

tempo_calculo += omp_get_wtime() - inicio;
```

Esse tempo inclui o custo do paralelismo dentro de `compute_next_state`, como a entrada e a saída da região paralela, o escalonamento e a sincronização. Não inclui a inicialização dos campos, a alocação, os diagnósticos, a gravação de CSVs ou a geração do GIF.

O argumento `-benchmark` define `save_every=0`, desativando frames, metadados de visualização e diagnósticos periódicos. Ele mantém a mesma evolução numérica. A saída ainda informa o tempo de cálculo para que o script o extraia.

4 Metodologia experimental

4.1 Ambiente

O hardware foi informado como o mesmo utilizado na Tarefa 10. A versão do compilador foi confirmada para os testes da Tarefa 11.

Tabela 1: Ambiente informado para as medições.

Componente	Especificação
Processador	Intel Core i5-6400, frequência nominal de 2,70 GHz
Núcleos / threads de hardware	4 núcleos físicos / 4 threads
Cache L1d / L1i	128 KiB / 128 KiB no total, quatro instâncias de cada
Cache L2 / L3	1 MiB / 6 MiB no total
Memória RAM	Aproximadamente 24 GiB
Sistema	Ubuntu Linux
Compilador	GCC 13.3.0, pacote Ubuntu `13.3.0-6ubuntu2~24.04.1`
Opções	`-O2 -fopenmp`, ligação com `-lm`

Não foram registrados controles de frequência da CPU, carga de outros processos ou afinidade das threads. A frequência nominal não significa frequência constante durante as execuções. Essas condições são consideradas na interpretação da variabilidade.

4.2 Duas cargas com finalidades diferentes

A visualização utiliza os parâmetros padrão. O desempenho utiliza uma carga maior para reduzir a influência relativa de medições muito curtas.

Tabela 2: Parâmetros das duas cargas analisadas.

Parâmetro	Validação visual	Desempenho
Modo	2	2
Malha	80×80	400×400
Passos	1.000	5.000
$\Delta x = \Delta y$	0,0125	0,0025
Δt	0,003125	0,000125
Tempo físico final	3,125	0,625
Atualizações de células	$6{,}4 \times 10^6$	$8{,}0 \times 10^8$
Frames	101, a cada 10 passos	Desativados

O segundo caso realiza 125 vezes mais atualizações de células, mas simula um tempo físico menor devido à redução de Δt. O tempo físico da tabela não é o tempo medido pelo relógio do computador.

4.3 Configurações e estatísticas

Foram comparadas duas versões, três quantidades de threads solicitadas $(1, 2, 4)$ e três schedules, totalizando 18 configurações. Cada configuração foi executada três vezes, gerando 54 medições:

Código 3: Comando que corresponde à configuração registrada nos resultados.

```
bash executar_testes.sh 2 3 400 400 5000
```

O script define `OMP_DYNAMIC=FALSE`, solicita o número de threads com `OMP_NUM_THREADS` e aplica cada schedule por `OMP_SCHEDULE`. Ele executa as configurações em ordem fixa, sem uma etapa de aquecimento separada. O número efetivo de threads não é gravado nos CSVs.

Para três tempos T_1, T_2, T_3, calculam-se

$$\overline{T} = \frac{1}{3}\sum_{k=1}^{3} T_k, \qquad s = \sqrt{\frac{1}{2}\sum_{k=1}^{3}(T_k - \overline{T})^2}, \qquad CV = 100\frac{s}{\overline{T}}.$$

As médias e os speedups são produzidos pelo script original. O desvio padrão amostral e o CV foram calculados adicionalmente para este relatório a partir dos mesmos tempos brutos. Nenhuma medição foi removida.

A referência comum é a média da versão normal com uma thread e `static`:

$$T_{\text{ref}} = 5{,}755341 \text{ s}, \qquad S = \frac{T_{\text{ref}}}{\overline{T}}.$$

Esse speedup mede a vantagem sobre uma configuração OpenMP de uma thread, e não sobre um executável sequencial compilado sem OpenMP. A pasta `Old/` foi usada como referência de correção.

5 Validação numérica

Nos modos parado e uniforme, as diferenças entre valores vizinhos são nulas. A Equação 3 preserva esses campos. Na verificação da implementação em ambiente de desenvolvimento, os 101 frames dos três modos foram comparados com os da versão sequencial de `Old/`; as versões normal e com `collapse(2)` produziram arquivos idênticos para os parâmetros padrão.

Os resultados de campo enviados ao repositório correspondem ao modo 2 na malha de 80×80. Os 101 frames dessa execução também coincidem, byte a byte, com os arquivos de referência em `Old/`. A análise desses frames não encontrou valores não finitos.

5.1 Decaimento e conservação da média

Para medir a perturbação no modo 2, utiliza-se $u_{\mathrm{ref}} = 1$ e $v_{\mathrm{ref}} = 0$:

$$A_{\max} = \max_{i,j} \sqrt{(u_{i,j} - 1)^2 + v_{i,j}^2}, \qquad A_{\mathrm{RMS}} = \sqrt{\frac{1}{N_x N_y} \sum_{i,j} \left[(u_{i,j} - 1)^2 + v_{i,j}^2 \right]}.$$

Tabela 3: Medidas extraídas dos frames inicial e final enviados ao repositório.

Medida	Inicial	Final
Máximo da perturbação	0,496958	0,046535
RMS da perturbação	0,070898	0,022755
Média de u	1,020106193	1,020106193
Variância de u	0,004622289	0,000113514
Máximo de $\lvert v \rvert$	0	0

O máximo e o RMS não crescem entre frames consecutivos, e a variância diminui. A maior variação da média de u ao longo dos arquivos é inferior a 4×10^{-14}, compatível com arredondamento numérico e a precisão dos CSVs.

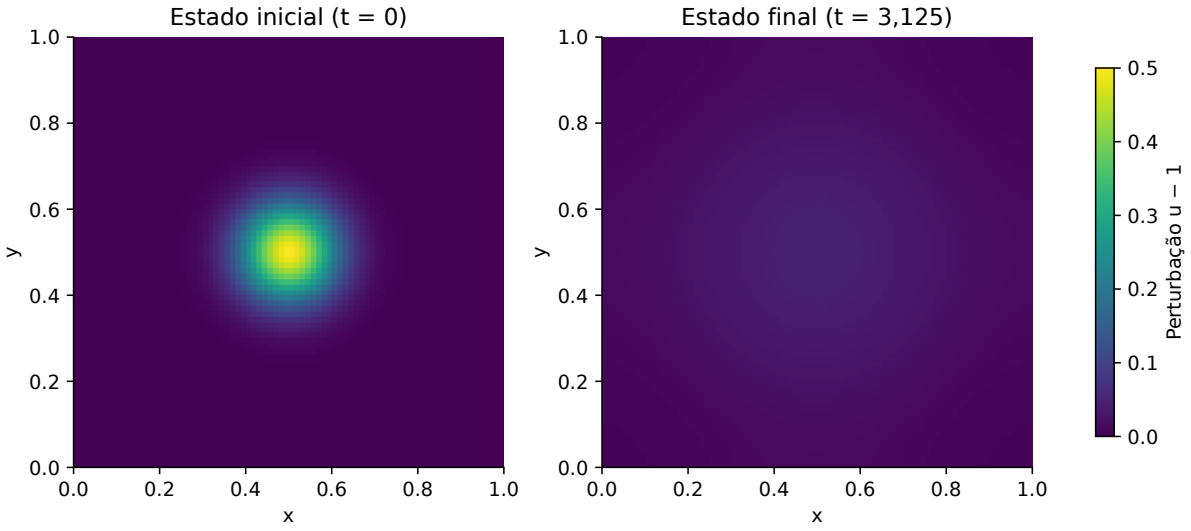


Figura 1: Perturbação inicial e final na mesma escala de cores, na malha de 80×80. A redução do pico e o espalhamento são compatíveis com a difusão.

Sob contornos periódicos, as contribuições dos vizinhos no Laplaciano se cancelam quando somadas sobre todo o domínio. Por isso, a média é conservada. A gaussiana inicial é positiva e acrescenta aproximadamente 0,020106193 à média de u; a tendência de longo prazo é um campo uniforme próximo de $u = 1{,}020106193$, e não exatamente $u = 1$.

Assim, o RMS medido em relação a $u = 1$ não precisa tender a zero. A variância em relação à própria média é uma medida complementar da uniformização. O estado final observado ainda não é perfeitamente uniforme.

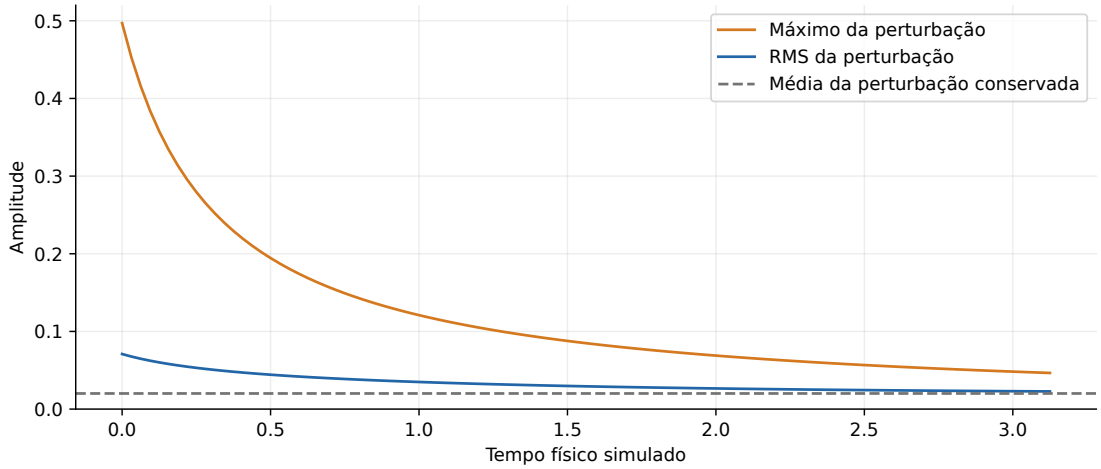


Figura 2: Decaimento do máximo e do RMS nos 101 frames. A linha tracejada representa a média positiva da perturbação que é conservada.

A concordância com `Old/` verifica a preservação do comportamento na paralelização. Ela não substitui um estudo de convergência de malha ou a comparação com uma solução analítica, que não fazem parte destes resultados.

6 Resultados de desempenho

A Tabela 4 reúne as 18 configurações. A diferença entre os tempos das repetições é mantida no desvio padrão, inclusive nos casos com maior dispersão.

Tabela 4: Resultados da carga de 400 × 400 células e 5.000 passos. s é o desvio padrão amostral; CV é o coeficiente de variação.

Versão	Schedule	Threads	Média (s)	s (s)	Speedup	CV (%)
Normal	`static`	1	5,755	0,026	1,000	0,45
Normal	`dynamic,8`	1	5,595	0,123	1,029	2,21
Normal	`guided,8`	1	5,451	0,095	1,056	1,74
Normal	`static`	2	2,899	0,029	1,985	1,01
Normal	`dynamic,8`	2	2,907	0,020	1,980	0,68
Normal	`guided,8`	2	2,801	0,020	2,055	0,73
Normal	`static`	4	2,054	0,420	2,801	20,43
Normal	`dynamic,8`	4	1,777	0,071	3,239	3,99
Normal	`guided,8`	4	1,762	0,038	3,267	2,15
Collapse	`static`	1	6,041	0,057	0,953	0,95
Collapse	`dynamic,8`	1	9,759	0,015	0,590	0,16
Collapse	`guided,8`	1	6,099	0,028	0,944	0,45
Collapse	`static`	2	3,211	0,040	1,792	1,23
Collapse	`dynamic,8`	2	9,505	0,118	0,606	1,25
Collapse	`guided,8`	2	3,155	0,040	1,824	1,28
Collapse	`static`	4	2,225	0,347	2,587	15,59
Collapse	`dynamic,8`	4	6,890	0,226	0,835	3,28
Collapse	`guided,8`	4	1,943	0,024	2,962	1,25

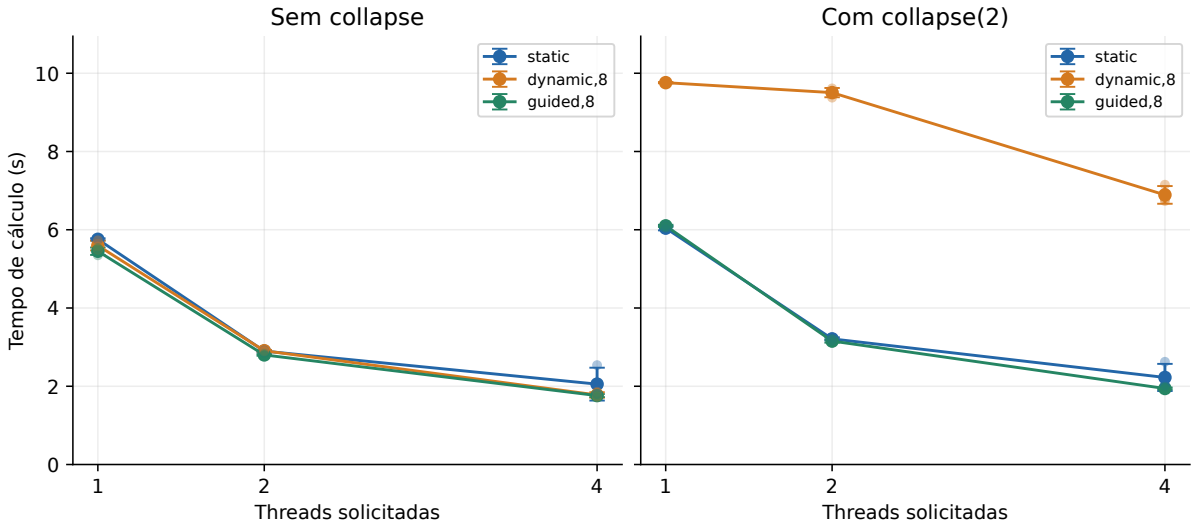


Figura 3: Tempos médios por configuração. As barras indicam um desvio padrão amostral e os pontos menores mostram as três medições. Não são intervalos de confiança.

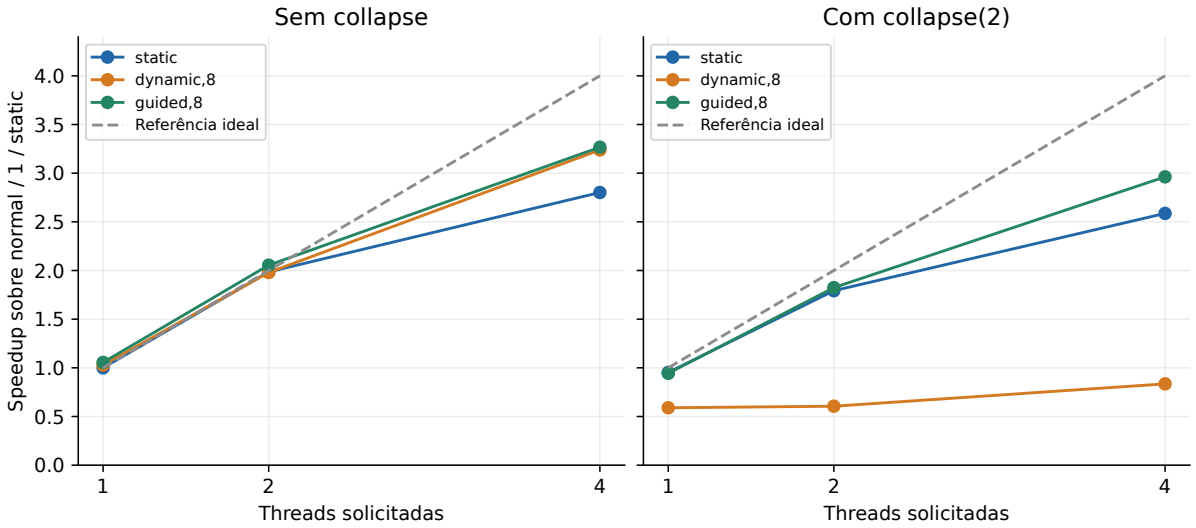


Figura 4: Speedup em relação à mesma referência normal / uma thread / `static`. A linha ideal é uma referência de escala, não um resultado medido.

O menor tempo médio foi obtido pela versão normal com quatro threads e `guided,8`: 1,761822 s, com speedup de aproximadamente 3,267. Com `dynamic,8`, a mesma versão obteve 1,777005 s e speedup de 3,239.

A melhor configuração com `collapse(2)` também utilizou quatro threads e `guided,8`, com média de 1,943000 s. Entre as médias deste experimento, nenhuma configuração com `collapse(2)` superou a configuração normal de mesmo schedule e mesma quantidade de threads.

7 Discussão

7.1 Escalabilidade e referência do speedup

Sem `collapse`, `static` passou de 5,755341 s com uma thread para 2,899031 s com duas, correspondendo a 1,985 vezes de aceleração. Esse resultado é próximo da redução ideal pela metade.

Para a melhor configuração de quatro threads, a razão $S/4$ é aproximadamente 81,7% em relação à referência comum. Esse percentual não deve ser confundido com utilização medida da CPU ou com eficiência relativa à versão sequencial de `Old/`.

O speedup 2,055 observado para duas threads com `guided,8` não comprova aceleração superlinear, pois a referência usa `static`. Ao comparar `guided,8` com seu próprio tempo de uma thread,

$$\frac{5{,}451358}{2{,}800924} \approx 1{,}946.$$

Portanto, diferenças entre schedules na configuração de uma thread também influenciam os speedups apresentados.

7.2 Por que `collapse(2)` não melhorou os tempos

A malha possui 400 linhas e foram solicitadas no máximo quatro threads. O laço externo já oferece iterações suficientes para a distribuição. Como o cálculo por célula é uniforme, aumentar o espaço lógico de 400 linhas para 160.000 células não cria, por si só, um benefício de balanceamento.

Além disso, o percurso em linhas respeita a disposição contígua dos dados. Alterar o particionamento pode modificar a localidade e os custos de execução. Os tempos confirmam a diferença de desempenho, mas, sem contadores de hardware, não permitem atribuir quantitativamente essa diferença a cache, divisão de índices ou largura de banda de memória.

O resultado não significa que `collapse` seja sempre inadequado. Ele pode ser útil quando o laço externo oferece poucas iterações em relação à quantidade de threads ou quando combinar as dimensões melhora a distribuição do trabalho.

7.3 Custo de `dynamic,8` com `collapse(2)`

Na versão normal, um bloco de oito iterações corresponde a oito linhas, ou 3.200 células. Por passo, as 400 linhas formam 50 blocos.

Com `collapse(2)`, as 160.000 células formam 20.000 blocos de oito células por passo. Ao longo de 5.000 passos, são aproximadamente 100 milhões de blocos, contra 250 mil na versão normal: 400 vezes mais unidades de distribuição.

Essa diferença de granularidade é compatível com o custo elevado observado. A versão com `collapse(2)` e `dynamic,8` levou 9,759254 s com uma thread e 6,890485 s com quatro. Há melhora ao aumentar suas próprias threads, mas a configuração de quatro threads continua mais lenta que a referência comum de uma thread.

O `guided,8` inicia com blocos maiores e reduz o número de obtenções de trabalho muito fino durante boa parte do laço. Isso é coerente com seus tempos menores. A explicação é baseada na política de distribuição e nas medições; não foi feito um perfil interno do runtime para separar cada custo.

7.4 Variabilidade e limites das conclusões

As configurações `static` com quatro threads apresentaram os maiores coeficientes de variação: 20,43% na versão normal e 15,59% com `collapse(2)`. Os tempos brutos foram, respectivamente,

$$(2{,}538934;\ 1{,}815439;\ 1{,}808791)\ \text{s}$$

e

$$(2{,}009164;\ 2{,}625166;\ 2{,}040834)\ \text{s}.$$

Os valores mais altos não foram descartados. Podem refletir condições do sistema, frequência ou escalonamento, mas o experimento não registrou evidências para determinar a causa.

A diferença entre `guided,8` e `dynamic,8` com quatro threads na versão normal é de aproximadamente 0,015 s, ou 0,85%. Como essa diferença é pequena diante da dispersão das três amostras, não há base suficiente para afirmar superioridade geral de um deles.

O uso de ordem fixa, a ausência de aquecimento separado e a pequena quantidade de repetições limitam a análise estatística. Ainda assim, as tendências mais marcantes, como o custo de `dynamic,8` com `collapse(2)`, aparecem de forma consistente nas amostras.

8 Conclusão

A implementação reproduziu o comportamento da versão sequencial nos modos analisados. Na execução com perturbação, o pico e o RMS diminuíram, a média foi conservada e a componente v permaneceu nula, em concordância com o modelo de difusão viscosa com contornos periódicos.

O aumento da carga permitiu obter tempos de segundos e observar aceleração mais clara. A versão normal atingiu speedup de aproximadamente 3,267 com quatro threads, enquanto `collapse(2)` não melhorou as médias nas condições avaliadas.

O principal aprendizado é que a escolha do schedule depende também da unidade de trabalho. O mesmo tamanho de bloco, oito, passou a significar oito células em vez de oito linhas com o uso de `collapse(2)`, aumentando muito o custo de distribuição dinâmica. Para este problema e esta quantidade de threads, paralelizar somente o laço externo foi suficiente.

As conclusões se restringem ao hardware, à carga e às amostras apresentados. Como continuidade, seria possível ampliar o número de repetições, controlar a afinidade das threads e variar o tamanho dos blocos dinâmicos, mantendo a mesma referência e a mesma carga.

A Reprodução das execuções

Os comandos a seguir são executados na pasta `Tarefa_11`. Para gerar os campos e o GIF:

```
gcc -O2 -fopenmp fluid_sim.c fluid.c -lm -o fluid_sim
OMP_NUM_THREADS=4 OMP_SCHEDULE=static ./fluid_sim 2

python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy matplotlib pillow
python visualize_fluid.py
```

O ambiente virtual é necessário apenas para as dependências Python. No Ubuntu, o pacote `python3-venv` deve estar instalado. O programa em C não depende de Python.

Para repetir a comparação de desempenho:

```
bash executar_testes.sh 2 3 400 400 5000
```

O script recompila as duas versões e grava `tempos.csv`, `resumo.csv` e `configuracao.txt` em `resultados_openmp/`. Uma nova execução substitui esses arquivos.

Para compilar manualmente a versão com `collapse(2)` e testar uma configuração:

```
gcc -O2 -fopenmp -DUSE_COLLAPSE \
    -DNX=400 -DNY=400 -DSTEPS=5000 \
    fluid_sim.c fluid.c -lm -o fluid_collapse
OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 OMP_SCHEDULE=guided,8 \
    ./fluid_collapse 2 --benchmark
```

B Arquivos que acompanham o relatório

O projeto LaTeX é independente da posição do restante do repositório. A pasta `dados/` contém cópias dos três arquivos de desempenho, metadados da visualização, os campos inicial e final, a série de diagnósticos dos 101 frames e as estatísticas derivadas.

A pasta `codigos/` contém cópias dos cinco fontes analisados. Elas documentam a revisão usada neste relatório e não substituem os fontes principais da tarefa. A função `compute_next_state`, em `fluid.c`, concentra o cálculo paralelo; `run_simulation` delimita a medição, e `executar_testes.sh` define a comparação.

O arquivo `gerar_figuras.py` recalcula os gráficos e a tabela a partir dos dados incluídos, sem executar uma nova simulação. As figuras e a tabela já estão prontas, de modo que a compilação do `main.tex` não exige Python, acesso à rede ou autorização de *shell escape*.

Referências

[1] OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface, Version 5.2*. 2021. Disponível em: `https://www.openmp.org/specifications/`.

[2] PINTO, Matheus Bezeril. *ProgParalela2026.2: Tarefa 11*. Código e resultados da revisão `5192499`, 2026. Disponível em: `https://github.com/39Matheus/ProgParalela2026.2/tree/5192499/Tarefa_11`.